МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра ИиСП

Отчет
по лабораторной работе № 3

по дисциплине «Машинно-зависимые языки программирования»
Вариант 4

Выполнил: ст. гр. ПС-13
Петялин А.Н.
Проверил: доцент кафедры ИиСП Баев А.А.

г. Йошкар-Ола
2025

Цель работы: протестировать некоторые арифметические и логические операции, условные переходы в режиме Debug и с использованием файла стимуляции на языке asm.

Задания на лабораторную работу:

Научиться пользоваться Atmel Studio

Научиться использовать файлы стимуляции

Написать алгоритм для проверки работы флагов у тех или иных команд

1. Теоретические сведения

Для ознакомления с режимом отладки создадим новый проект и напишем следующий шаблон.

```
reset:
        rjmp main

main:

loop:
        rjmp loop
```

Следующий пример должен работать с любым устройством ATtiny или ATmega, имеющим как минимум регистра порта B.

Он должен быть запущен с активным представлением ввода-вывода и настроен для вывод.

Эта программа настраивает PORTB в качестве вывода, а затем выполняет цикл, считывая все, что присутствует на PINB, увеличивает его на единицу и выводит в PORTB.

```
reset:
        rjmp main
main:
        ldi r16, 0xff
        out DDRB, r16
loop:
        in r15, PINB
        inc r15
        out PORTB, r15
        rjmp loop
```

Для перехода в режим отладки нажимаем Debug->Start Debugging and Break. При этом программа запустится и остановится на первой строке.

На вкладке Processor Status можно отслеживать Количество пройденных циклов, состояние Status Register, а также состояние регистров общего назначения.

На вкладке Registers также отображается состояние регистров общего назначения.

На вкладке IO можно отследить состояние регистров ввода/вывода (4).

Для перехода на следующую команду следует нажать клавишу F10.

При изменении значения поля, оно подсвечивается красным цветом. На следующем шаге значение r15 будет передано в регистр PORTB.

Однако для большого объема кода, особенно с задержкой, нажимать F10 будет неудобно (например, 4800000 раз). В этом случае можно поставить указатель на требуемую строку и выбрать в меню Run to cursor. В этом случае перейдём сразу на строку rjmp loop.

Альтернативным способом отследить состояния портов является использования файла стимуляции.

Для данного примера нам нужны регистры: PINB, PORTB.

Для этого добавим в проект новый файл с именем Asm_Lab_Math.stim. Также этот файл нужно указать в настройках проекта.

Файл .stim $log PORTB $log PINB $startlog Asm_Lab_Math_log_output.stim #102 $endrep $stoplog $break В данном случае выполняется логирование регистров PINB, PORTB, и запись их значений, при изменении, в выходной файл. Следует отметить, что логирование можно сделать только для регистров ввода-вывода. Запуск стимуляции выполняется в три шага:

1. Debug->Start Debugging and Break
2. Debug->Execute Stimulifile
3. Debug->Continue.

После этого можно заметить, что счетчик циклов равен 103.

В выходном файле #5 – это количество тактов до изменения отслеживаемого регистра, PORTB = 0x01 – значение регистра после изменения.

2. **Практическая часть**

Задание 1: Выполнить проверку воздействия различных операций на флаги регистра состояний.

Мнемоника	**Операнды**	**Описание**	**Операция**	**Флаги**	**Циклы**
FMUL	Rd, Rr	Умножение дробных чисел без знака	R1:R0=(Rd * Rr) << 1	Z, C	2

Программа:

```
reset:
        rjmp main
main:
        ldi r18, 0xFF
        ldi r19, 0xFF
        out OCR0A, r18
        out OCR0B, r19

loop:
        in r18, OCR0A
        in r19, OCR0B
        fmul r18, r19
        out OCR0A, r18
        out OCR0B, r19

        rjmp loop
```

.stim файл:

```
$log OCR0A
$log OCR0B
$log SREG
$startlog p1_1_result.stim
#5
OCR0A = 0
OCR0B = 0
#7
OCR0A = 15
OCR0B = 15
#7
OCR0A = 182
OCR0B = 181
#7
OCR0A = 20
OCR0B = 20
#7
$stoplog
$break
```

Выходной результат:

```
#3
OCR0A = 0xff
#1
OCR0B = 0xff
#2
```

OCR0B = 0x00

OCR0A = 0x00

#2

SREG = 0x02

#1

OCR0A = 0xff

#4

OCR0B = 0x0f

OCR0A = 0x0f

#3

SREG = 0x00

#5

OCR0B = 0xb5

OCR0A = 0xb6

#3

SREG = 0x01

#3

OCR0B = 0x14

OCR0A = 0x14

#5

SREG = 0x00

Результат:

	OCR0A	OCR0B	SREG	Флаги
1	0x00	0x00	0x02	Z
2	0x0F	0x0F	0x00	-
3	0xB6	0xB5	0x01	C

4	0x14	0x14	0x00	-

Мнемоника	Операнды	Описание	Операция	Флаги	Циклы
LSL	Rd	Логический сдвиг влево	Rd(n+1)=Rd(n), Rd(0)=0,C=Rd(7)	Z,C,N,V,H,S	1

Программа:

```
reset:
        rjmp main
main:
   ldi r18, 0x00
        out OCR0A, r18

loop:
        in r18, OCR0A
        lsl r18
        out OCR0A, r18

        rjmp loop
```

.stim файл:

```
$log OCR0A
$log SREG
$startlog p1_2.stim
#3
OCR0A = 0
#5
OCR0A = 1
```

#5

OCR0A = 255

#5

OCR0A = 127

#5

OCR0A = 2

#5

$stoplog

$break

Выходной результат:

#4

SREG = 0x02

#4

OCR0A = 0x01

#1

SREG = 0x00

#1

OCR0A = 0x02

#3

OCR0A = 0xff

#1

SREG = 0x35

#1

OCR0A = 0xfe

#3

OCR0A = 0x7f

#1

SREG = 0x2c

#1

OCR0A = 0xfe

#3

OCR0A = 0x02

#1

SREG = 0x00

#1

OCR0A = 0x04

Результат:

	OCR0A	OCR0A’	SREG	Флаги
1	0x00	0x00	0x02	Z
2	0x01	0x02	0x00	-
3	0xFF	0xFE	0x35	H, S, N, C
4	0x7F	0xFE	0x2c	H, V, N
5	0x02	0x04	0x00	-

Мнемоника	**Операнды**	**Описание**	**Операция**	**Флаги**	**Циклы**
ADD=	Rd, Rr=	Суммирование без переноса	Rd = Rd + Rr=	Z,C,N,V,H,S=	1

Программа:

```
reset:
        rjmp main
main:
        ldi r18, 0xFF
```

```
        ldi r19, 0xFF
        out OCR0A, r18
        out OCR0B, r19
loop:
        in r18, OCR0A
        in r19, OCR0B
        add r18, r19
        out OCR0A, r18
        out OCR0B, r19

        rjmp loop
```

.stim файл:

```
$log OCR0A
$log OCR0B
$log SREG
$startlog p1_3.stim
#5
OCR0A = 0
OCR0B = 0
#7
OCR0A = 1
OCR0B = 1
#7
OCR0A = 255
OCR0B = 255
#7
OCR0A = 1
```

OCR0B = 2

#7

OCR0A = 64

OCR0B = 64

#7

OCR0A = 3

OCR0B = 4

#7

$stoplog

$break

Выходной результат:

#3

OCR0A = 0xff

#1

OCR0B = 0xff

#1

OCR0B = 0x00

OCR0A = 0x00

#2

SREG = 0x02

#5

OCR0B = 0x01

OCR0A = 0x01

#2

SREG = 0x00

#1

OCR0A = 0x02

```
#4
OCR0B = 0xff
OCR0A = 0xff
#2
SREG = 0x35
#1
OCR0A = 0xfe
#4
OCR0B = 0x02
OCR0A = 0x01
#2
SREG = 0x00
#1
OCR0A = 0x03
#4
OCR0B = 0x40
OCR0A = 0x40
#2
SREG = 0x0c
#1
OCR0A = 0x80
#4
OCR0B = 0x04
OCR0A = 0x03
#2
SREG = 0x00
#1
OCR0A = 0x07
```

Результат:

	OCR0A	OCR0B	OCR0A’	SREG	Флаги
1	0x00	0x00	0x00	0x02	Z
2	0x01	0x01	0x02	0x00	-
3	0xFF	0xFF	0xFE	0x35	H, S, N, C
4	0x01	0x02	0x03	0x00	-
5	0x40	0x40	0x80	0x0C	V, N
6	0x03	0x04	0x07	0x00	-

Мнемоника	**Операнды**	**Описание**	**Операция**	**Флаги**	**Циклы**
OR	Rd, Rr	Логическое ИЛИ	Rd = Rd V Rr	Z,N,V,S	1

Программа:

```
reset:
        rjmp main
main:
        ldi r20, 0x40
        ldi r21, 0x40
        add r20, r21
        ldi r18, 0xFF
        ldi r19, 0xFF
        out OCR0A, r18
        out OCR0B, r19
loop:
```

```
        in r18, OCR0A
        in r19, OCR0B
        or r18, r19
        out OCR0A, r18
        out OCR0B, r19

        rjmp loop
```

В main мы включаем флаг V, чтобы доказать влияние команды OR на него.

.stim файл:

```
$log OCR0A
$log OCR0B
$log SREG
$startlog p1_4.stim
#8
OCR0A = 0
OCR0B = 255
#7
OCR0A = 10
OCR0B = 10
#7
OCR0A = 0
OCR0B = 0
#7
OCR0A = 1
OCR0B = 1
#7
```

```
$stoplog
$break
```

Выходной результат:

```
#3
SREG = 0x0c
#3
OCR0A = 0xff
#1
OCR0B = 0xff
#1
OCR0A = 0x00
#2
SREG = 0x14
#1
OCR0A = 0xff
#4
OCR0B = 0x0a
OCR0A = 0x0a
#2
SREG = 0x00
#5
OCR0B = 0x00
OCR0A = 0x00
#2
SREG = 0x02
#5
```

OCR0B = 0x01

OCR0A = 0x01

#2

SREG = 0x00

Результат:

	OCR0A	OCR0B	OCR0A’	SREG	Флаги
1	0x00	0xFF	0xFF	0x14	S, N
2	0x0A	0x0A	0x0A	0x00	-
3	0x00	0x00	0x00	0x02	Z
4	0x01	0x01	0x01	0x00	-

Флаг V выключается командой OR.

Мнемоника	**Операнды**	**Описание**	**Операция**	**Флаги**	**Циклы**
LSR	Rd	Логический сдвиг вправо	Rd(n)=Rd(n+1), Rd(7)=0,C=Rd(0)	Z,C,N,V,S	1

Программа:

```
reset:
        rjmp main

main:
        ldi r18, 0xFF
        out OCR0A, r18
        bset 2
```

```
loop:
        in r18, OCR0A
        lsr r18
        out OCR0A, r18

        rjmp loop
```

В main я вручную установил флаг N в SREG, чтобы показать, что команда LSR очищает его.

.stim файл:

```
$log OCR0A
$log SREG
$startlog p1_5.stim
#4
OCR0A = 1
#5
$stoplog
$break
```

Выходной результат:

```
#2
OCR0A = 0xff
#1
SREG = 0x04
#1
OCR0A = 0x01
```

#1

SREG = 0x1b

#1

OCR0A = 0x00

Результат:

	OCR0A	OCR0A’	SREG	Флаги
1	0x01	0x00	0x1B	S, V, Z, C

Флаг S устанавливается, если установлен флаг V, иначе очищается.

Флаг V устанавливается, если C = 1 и N = 0.

Флаг также устанавливается в случае, если флаг C = 0 и N = 1, но данная команда всегда очищает флаг N, следовательно флаг устанавливается только в случае, если установлен флаг C, иначе очищается.

Флаг N всегда очищается, так как при каждом сдвиге в начале будет 0, а следовательно число всегда остаётся положительным.

Флаг C устанавливается, если был установлен LSB, иначе очищается.

Задание 2: Написать программу с использованием условного перехода.

Мнемоника	**Операнды**	**Описание**	**Операция**	**Флаги**	**Циклы**
CPC	Rd, Rr	Сравнить с переносом	Rd - Rr - C	Z,C,N,V,H,S	1

Программа:

```
reset:
        rjmp main


main:
```

```
        ldi r18, 0xFF
        ldi r19, 0xFF
        out OCR0A, r18
        out OCR0B, r19
        bset 1

loop:
        in r18, OCR0A
        in r19, OCR0B
        cpc r18, r19

        rjmp loop
```

В main я вручную включил флаг Z,чтобы показать, что флаг Z сохраняет своё значение, если результатом команды был 0, иначе очищается.

.stim файл:

```
$log OCR0A
$log OCR0B
$log SREG
$startlog p2_1.stim
#6
OCR0A = 128
OCR0B = 127
#7
OCR0A = 128
OCR0B = 128
#7
```

OCR0A = 128

OCR0B = 129

#7

$stoplog

$break

Выходной результат:

#3

OCR0A = 0xff

#1

OCR0B = 0xff

#1

SREG = 0x02

#1

OCR0B = 0x7f

OCR0A = 0x80

#2

SREG = 0x38

#5

OCR0B = 0x80

#5

SREG = 0x00

#3

OCR0B = 0x81

#2

SREG = 0x35

Результат:

	OCR0A	OCR0B	SREG	Флаги
1	0x80	0x7F	0xFF	H, S, V
2	0x80	0x80	0x00	-
3	0x80	0x81	0x35	H, S, N, C

Мнемоника	**Операнды**	**Описание**	**Операция**	**Флаги**	**Циклы**
SBRS	Rr, b	Пропустить если бит в регистре установлен	if(Rr(b)==1) PC = PC + 2 or 3	None	1/2/3

Программа:

```
reset:
        rjmp main


main:
        ldi r18, 0x01
        out OCR0A, r18
        ldi r19, 0x00
        out OCR0B, r19
loop:
        in r18, OCR0A
        sbrs r18, 1
        ldi r19, 0x10
        out OCR0B, r19
        in r18, OCR0A
        sbrs r18, 1
        ldi r19, 0xff
```

```
out OCR0B, r19
in r18, OCR0A
sbrs r18, 1
jmp 0x00
rjmp loop
```

В программе проверим условия пропуска команды sbrs, протестируем прибавление PC +1, PC+2, PC+3.

.stim файл:

```
$log OCR0A
$log OCR0B
$log SREG
$startlog p2_2.stim
#5
OCR0A = 1
#4
OCR0A = 2
#4
OCR0A = 254
#3
$stoplog
$break
```

Выходной результат:

```
#2
OCR0A = 0x01
#6
OCR0B = 0x10
```

#1

OCR0A = 0x02

#4

OCR0A = 0xfe

Результат:

	OCR0A	OCR0B	SREG	Флаги
1	0x01	0x10	0x00	-
2	0x02	0x10	0x00	-
3	0xFE	0x10	0x00	-

Команда не влияет на флаги регистра статусов SREG.

Мнемоника	**Операнды**	**Описание**	**Операция**	**Флаги**	**Циклы**
BRTS	k	Перейти если флаг T установлен	if(T==1) PC = PC + k + 1	None	1/2

Программа:

```
reset:
        rjmp main


main:
        ldi r18, 0x01
        out OCR0A, r18
        ldi r19, 0x00
        out OCR0B, r19
```

```
loop:
        in r18, OCR0A
        bst r18, 2
        brts 0
        ldi r19, 0xff
        out OCR0B, r19
        clt
        in r18, OCR0A
        bst r18, 1
        brts aftersecondtest

        rjmp loop

aftersecondtest:
        clt
        ldi r19, 0x00
        out OCR0B, r19
        in r18, OCR0A
        bst r18, 1
        brts afterthirdtest
        rjmp loop

afterthirdtest:
        ldi r19, 0xff
        out OCR0B, r19
        rjmp loop
```

Команда BRTS переходит по меткам, поэтому добавим новые метки и отобразим в OCR0B, что эти переходы действительно совершаются.

.stim файл:

```
$log OCR0A

$log OCR0B

$log SREG

$startlog p2_3.stim

#5

OCR0A = 1

#6

OCR0A = 11

#7

OCR0A = 254

#3

$stoplog

$break
```

Выходной результат:

```
#2

OCR0A = 0x01

#7

OCR0B = 0xff

#2

OCR0A = 0x0b

#1

SREG = 0x40

#3

SREG = 0x00
```

#2

OCR0B = 0x00

#1

OCR0A = 0xfe

#1

SREG = 0x40

SREG здесь меняется нами вручную командой BST, которая устанавливает флаг T, если в регистре установлен определенный бит.Значения в OCR0A подобраны так, чтобы первый переход не срабатывал, а остальные срабатывали.

Результат:

	OCR0A	OCR0B	SREG	Флаги
1	0x01	0xFF	0x00	-
2	0x0B	0x00	0x00	-
3	0xFE	0xFF	0x00	-

Мнемоника	**Операнды**	**Описание**	**Операция**	**Флаги**	**Циклы**
BRVC	k	Перейти если флаг переполнения очищен	if(V==0) PC = PC + k + 1	None	1/2

Программа:

```
reset:
        rjmp main
```

```
main:
        ldi r18, 0x01
        out OCR0A, r18
        ldi r19, 0x00
        out OCR0B, r19
loop:
        in r18, OCR0A
        inc r18
        brvc afterfirsttest
        rjmp loop

afterfirsttest:
        ldi r19, 0xff
        out OCR0B, r19
        in r18, OCR0A
        inc r18
        brvc aftersecondtest
        ldi r19, 0x00
        out OCR0B, r19
        rjmp loop

aftersecondtest:
        rjmp loop
```

.stim файл:

```
$log OCR0A
$log OCR0B
```

$log SREG

$startlog p2_4.stim

#5

OCR0A = 1

#6

OCR0A = 127

#7

$stoplog

$break

Выходной результат:

#2

OCR0A = 0x01

#8

OCR0B = 0xff

#1

OCR0A = 0x7f

#1

SREG = 0x0c

#3

OCR0B = 0x00

Результат:

	OCR0A	OCR0B	SREG	Флаги
1	0x01	0xFF	0x00	-
2	0x7F	0x00	0x0C	V

Во втором случае мы включили флаг V с помощью команды inc к регистру OCR0A, чтобы показать, как работает команда BRVC.

Мнемоника	**Операнды**	**Описание**	**Операция**	**Флаги**	**Циклы**
SBRC	Rr, b	Пропустить если бит в регистре очищен	if(Rr(b)==0) PC = PC + 2 or 3	None	1/2/3

Программа:

```
reset:
        rjmp main


main:
        ldi r18, 0x00
        out OCR0A, r18
        ldi r19, 0x01
        out OCR0B, r19



loop:
        in r18, OCR0A
        sbrc r18, 0
        ldi r19, 0x00
        out OCR0B, r19
        in r18, OCR0A
        sbrc r18, 0
        nop
        ldi r19, 0xff
```

```
        out OCR0B, r19
        in r18, OCR0A
        sbrc r18, 0
        jmp 0x00
        rjmp loop
```

.stim файл:

```
$log OCR0A
$log OCR0B
$log SREG
$startlog p2_5.stim
#5
OCR0A = 1
#4
OCR0A = 2
#5
OCR0A = 4
#7
$stoplog
$break
```

Выходной результат:

```
        #4
        OCR0B = 0x01
        #1
        OCR0A = 0x01
        #3
        OCR0B = 0x00
```

#1

OCR0A = 0x02

#4

OCR0B = 0xff

#1

OCR0A = 0x04

#5

OCR0B = 0x00

Результат:

	OCR0A	OCR0B	SREG	Флаги
1	0x01	0x00	0x00	-
2	0x02	0xAA	0x00	-
3	0x04	0x00	0x00	-

В первом случае со значением 0x01 в OCR0A условие команды sbrc не выполняется, во втором случае пропускается команда в одно слово, в третьем – команда jmp.

Задание 3. Работа с массивами

Выводы

Я узнал, как различные команды влияют на флаги и впринципе роль флагов в работе программы. Я научился пользоваться Atmel Studio.